Individual Assessment Coversheet

To be attached to the front of the assessment.

Campus: Tyger Valley

Faculty: Information Technology

Module Code: ITSEA2-12

Group: 2

Lecturer's Name: Sandura Shumba

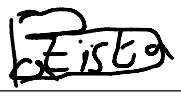
Student Full Name: Keenan Paiva Batista

Student Number: 4845350

Indicate	Yes	No
Plagiarism report attached		

Declaration:

I declare that this assessment is my own original work except for source material explicitly acknowledged. I also declare that this assessment or any other of my original work related to it has not been previously, or is not being simultaneously, submitted for this or any other course. I am aware of the AI policy and acknowledge that I have not used any AI technology to generate or manipulate data, other than as permitted by the assessment instructions. I also declare that I am aware of the Institution's policy and regulations on honesty in academic work as set out in the Conditions of Enrolment, and of the disciplinary guidelines applicable to breaches of such policy and regulations.

Signature	Date
	13 June 2025

Lecturer's Comments:

Marks Awarded:	%

Signature	Date

Table of Contents

Section A

Section A

Question 1

1.1) Performing acceptance tests by providing software deliverables to the system customers enables them to define the criteria that the system must meet before it can be deployed for operational use, which can increase the overall reliability of the software than without this form of testing (Leung and Wong, 1997). Implementing acceptance testing can be beneficial for identifying potential software defects and inefficiencies, because system customers can detect and report aspects of the system that does not meet their requirements or can negatively impact their operations. System developers can utilise this feedback to make the system suitable for the users and their business environment. This decreases the likelihood of a potential failure from occurring, because the system can effectively perform the user's operations as opposed to a system that was not designed to handle these operations. This demonstrates that acceptance testing can improve the reliability of the software as it enables system developers to create the system that can perform all the operations required from the system customers.

Therefore, many of the issues that the financial institution faced would have been minimised or prevented if acceptance testing was performed. Acceptance testing would have decreased the confusion that system developers could face by providing a consistent and structured approach for testing the system's reliability. This decrease in confusion can also reduce the amount of potential human errors from occurring during the development of the system. Additionally, acceptance testing can be performed throughout the various stages of the system's development by providing software deliverables to the users. This agile approach to testing the system allows the developers to solve defects in timely manner without it accumulating as well as reduce the chances of any defects being overlooked. Solving these defects in the early stages of the system's development can also prevent additional defects from being introduced through already-existing issues.

1.2) Equivalence partitioning is a software testing technique that identifies partitions, which are made up of groups of input or outputs. These partitions that are identified when performing equivalence partitioning will have at least one test case applied to it, which improves the financial institution's selection of test cases due to their being more test cases present during software testing (Anonymous, 2009). Test case selection can be further improved upon in equivalence partitioning, because there are guidelines that can be followed to assist developers when choosing test cases. Additionally, equivalence partitioning can also help the financial institution detect additional defects as mistakes that are often made by the developers at the edges of the identified partitions can be detected through this technique (Ian Sommerville, 2016).

Moreover, boundary value analysis is a software testing technique that is used with equivalence partitioning to determine the boundaries of each input and output partition where values inside and outside the boundary are tested. Boundary values analysis can improve defect detecting in the financial institution, because defects usually occur near the boundaries that are identified where testers can then identify these defects (Anonymous, 2009). In addition, these defects that tend to be found near these boundaries also increases the amount of test cases that can occur than usual as testers will perform tests over the boundaries to determine if there are defects, the increase in test cases allows for test case selection to improve as there are more test cases to select from.

1.3) Test-First Development (TFD) is primarily known as Test-Driven Development (TDD) where the prior term is used in scenarios where the distinction of the test-last practice of software development is emphasised, therefore TDD is the more appropriate term for this scenario since performing tests at the end of software development was detrimental (Stejskal, 2014). TTD is an approach in software development where code is development incrementally where each code increment has to pass its corresponding test in order start with the next code increment. The TDD method introduces multiple benefits for software development such as the improvement in the quality and readability of the code due to developers performing refactoring for each code increment and the capability for regression testing, because the created unit tests enables developers perform them multiple times at each increment (Mafi and Mirian-Hosseinabadi, 2024).

Implementing the TDD in this scenario can greatly improve the financial institution's ability to detect and remove software defects due to consistent approach that TDD provides. Utilising TDD, where aspects of the software are incrementally created and tested before moving on to the next aspect of the system, provides developers and testers with a clear testing structure that can be followed to reduce the possibility of human errors due to the multiple, inconsistent approaches.

Moreover, this clear structure also enables them to detect and solve defects in the software in an efficient manner, because defects are caught in the early stages of software development without it accumulating over a long period of time where it will become difficult to remove it. In addition, TDD enables the developers and testers to implement regression testing at each increment during software development due to the ability for unit tests to be reused. Regression testing allows the testers to ensure that each code increment has not indirectly re-introduced defects in previous code increments, preventing these defects for accumulating and improving the overall quality of the software.

1.4) Manual and automated testing are both processes that are used to detect defects within software. Manual testing is conducted through testers who manually search for defects, while automated testing utilises reusable, programmed scripts to execute multiple test cases for the software. Automated testing can cover a larger test coverage in a shorter amount of time than manual testing, however the test suite must be eligible for automated testing (Bindu Bhargavi and Suma, 2022). Despite the benefits that automated testing introduces, manual testing can be more valuable than automated testing when aspects of the software cannot be tested with a programmed script or it ends up being more cost effective to test it manually than automatically.

Reusable automated tests can be utilised by this financial institution throughout the development of their online banking system for various reasons such as to detect defects that were overlooked when manually testing the system and reduce the chances for human errors. Additionally, automated testing can also be used to improve the efficiency at detecting vast amounts of defects due to the ability for the financial institution to execute multiple programmed scripts. However, manual testing can be utilised to test aspects of the software in order to discover unknown defects that were missed during automated tests. These unknown defects are more likely to be found through manual testing than automated testing, because programmed scripts in automated testing only test for defects that the developers are aware of.

1.5) Release testing refers to a process where the release of a system is tested to determine if it meets functionality that system customers want. By implementing this testing strategy, the financial institution could have utilised it to determine if all system customer requirements have been met when the system has been developed, if these requirements have been met then the software should be ready to be released to the system customers. However, if these requirements are not met then the financial institution will have to make various alterations to the system until it

meets those requirements. Performing these tests would have made the system suitable for the system customers, increasing overall customer satisfaction.

In addition, system testing refers to a process where components of a system are integrated together in order to test the interaction between these components and detect any defects that occur during these interactions. The financial institution could have utilised this type of testing during the development of their software to detect and fix any new defects that are introduced. Detecting these defects can greatly reduce the overall quantity of overlooked defects in the system, preventing potential reputational damage and financial penalties.

1.6) Software quality assurance refers to the method of ensuring the quality of software, where the quality of the software depends on various aspects such as the lack of flaws that can compromise a system or create inefficiencies as well as customer satisfaction (Hassan, Mubashir, Shabir, Ullah, 2018).

Development testing in the context of software quality assurance refers to testing software throughout its development in order to detect and remove defects that compromises the quality of the software. These defects that can be identified during these tests can include issues that negatively impacts the security of the system and its users as well as the performance of the system. Identifying and fixing these issues will greatly improve the quality of the software as the security and efficiency will not be compromised.

However, user testing in the context of software quality assurance refers to users determining if the software meets the requirements of their business environment. Any aspects of the software that is missing or can be improved upon can be brought to the attention of the developers by the users during the process of user testing. The software can then be developed to meet all user requirements, which improves the overall quality of the software as the customer will be more satisfied with software that fits their environment instead of software that does not fit their environment.

Question 2

2.1) The KCSAP applied requirements engineering to determine the requirements, technical and social goals of the system before development began. These requirements and goals were identified by collaborating with individuals who will be impacted and make use of the system including agricultural specialists as well as local community members. Establishing communication between the engineers and local communities also enables them to identify the non-functional requirements of the system that needs to be taken into account during the life cycle of the system.

In addition, verification and validation was also applied by the KCSAP to ensure that system that was being developed functioned effectively and met user requirements. Verification can be seen within the scenario when the engineers designed the system in a manner where environmental constraints are mitigated to ensure that the stakeholders, like the farmers and organisations, receive important information as soon as possible. This was accomplished by utilising a distributed system with local nodes to counteract inconsistent internet connectivity and varying climate conditions of each Kenya region. The established communication between the engineers and the users of the system ensures that system validation can take place where the users decide if the system is suitable for them by determining if the systems matches their tasks, goals and responsibilities (Pandey, 2013).

Furthermore, deployment and maintenance is also performed by the KCSAP in this scenario when the system is designed to provide fault tolerance. Designing and deploying the system as a distributed system where each local node is isolated from each other to prevent a single node failure from impacting the whole network. The engineers after being notified of a node failure can then perform maintenance on that specific node that experienced a failure and make it functional. The engineers can also perform maintenance on the system as a whole by adding new services that the stakeholders request for or fixing vulnerabilities that are discovered.

2.2) A real-time system refers to a system where it performs operations based on information and results it receives from different hardware components as well as how long it takes for this information to be processed. This system can be made to make decisions and react according to its environment or control specialised hardware. Real-time systems can be categorised as either a soft or hard-real time system where each category can be used for specific situations.

A hard real-time system is a real-time system where response times have to meet constraints that are defined. If the system fails to meet these time constraints when performing operations, a system failure will occur. However, soft real-time systems will not fail if time response constraints are not met like hard real-time systems. Instead of experiencing a system failure, the performance of soft real-time systems will start to degrade when the time(Schoch and Laplante, 1995).

The features associated with KCSAP's real-time system are more aligned with hard than soft real-time constraints due to the amount of potential data being processed. Soft deadlines are often missed in real-time systems that utilise IoT devices, because the system is bombarded with periodic sensing data (Malik *et al.*, 2019). Utilising a soft real-time system within this scenario leads to the possibility that important sensing data that can be used to warn farmers of events like storms and pest outbreaks could be missed due to the sheer quantity of data that the system senses through the IoT components. The KCSAP's system will experience this problem, because it was designed to continuously monitor its environment for the signs of potential disasters such as the soil moisture levels and the weather.

A hard real-time system ensures that the most important sensing data that is identified is prioritised above the rest of the data that is sensed. Prioritising the most important data increases the likelihood that the system can adequately send a warning message to the farmers as soon as possible than not prioritising this data, allowing them to prepare for catastrophic events. The severity of a catastrophic event that occurs can be minimised or fully mitigated when farmers have enough preparation time in comparison to a soft real-time system sending the warning message at a later time.

2.3) A distributed system refers to a collection of software and hardware that appears as a single system to users. The core characteristics of distributed systems that should be taken into consideration are transparency, to what extent should it appear as a single system, openness, how each component can interact with each other, scalability, how can the system scale up or down, security, how the system is protected against attackers, quality of service, how the services are viewed as acceptable by the users and failure management, how failures are detected and contained to protect other components of the system.

Transparency is showcased in the KCSAP's distributed system when the farmers receive notifications on different environmental aspects that can prove to be valuable for their operations. These aspects can include current weather forecasts, soil moisture levels and pest outbreaks where each aspect is recorded by various hardware that the farmers are then made aware of when they receive messages regarding these aspects. Additionally, openness is a characteristic that is required for the system to function, because the sensor components are diverse to each other in order to meet the different climate risks and soil types across each Kenya region. Communication and distribution protocols should be utilised to ensure that each sensor can communicate over the distributed system after obtaining input from each separate environment (Csuhaj-Varjú and Vaszil, 2024). Scalability is also shown in this system by utilising local IoT devices across Kenya with cloud-based services that assists with scaling the system up despite having remote areas with inconsistent network connectivity. Quality of service can be identified by the response time of the system when it records and sends critical information to the farmers, enabling them to effectively prepare for potential catastrophic events such as storms. Moreover, failure management can be seen due to the distributed design of the system, because it enables for farmers to still receive information from other sensers if another sensor fails.

2.4) Service oriented architecture (SOA) refers to a strategic approach to software design where distributed systems are broken down into small, manageable components that can operate as stand-alone services. Systems that are designed with a SOA approach display characteristics of modularity, by decomposing the system into multiple services, interoperability, as it enables communication between disparate systems, service composition, by creating composite applications that are composed of various services, and scalability, due to the ability to decouple services and provide standardised interfaces (Fatima, Khtira, Asri, 2024).

Various organisations are able to effectively utilise these reusable services when performing their operations due to the modular and scalability aspects of SOA. Modularity ensures that the services that are created by breaking down KCSAP's system are fully operational by themselves, which enables organisations to alter these services with minimal effort to match their specific business operations. Moreover, organisations can then use service composition to combine multiple services together to further match their requirements such as an NGO requiring both weather advisory and crop disease services in order to effectively assist farmers.

Moreover, the standards introduced by SOA like WSDL can assist organisations with altering these services to match their needs by allowing them to define the service interface based on what it supports, how its accessed and where its located. The standards of SOA can also improve the

interoperability between each organisation that utilises KCSAP's distributed system like the SOAP standard. SOAP is a SOA standard that handles message exchanges in a manner that support communication between services, which can be useful for scenarios where NGOs and government agencies are able to utilise each other's services to effectively address environmental issues.

Question 3

3.1) The quality management process in the context of software development refers to conducting checks for each project deliverable to ensure that the organisation's standards and goals are met. The techniques utilised in the quality management process consist of creating a quality plan that defines the desired software qualities and the appropriate standards that should be followed as well as conducting reviews and inspections on the software and relevant documentation. The quality management process can ensure that InnovaPay is able to meet performance, security and reliability expectations by providing documentation that explains the initial requirements for the software and informing the software developers of issues that were identified in the software.

This initial documentation refers to the quality plan that specifies the desired software qualities and standards for the software that is being developed. This plan can improve the developer's understanding on the desired outcome of the software, which can reduce the number of potential issues that the developers have to address due to a misunderstanding of the software. Lessening the number of potential issues, increases the likelihood that the performance, security and reliability expectations will be met in short timeframe and not exceed the budget for the project than not lessening issues.

Moreover, performing reviews and inspections for each project deliverable can also increase the likelihood that the specified expectations are met by detecting potential defects and issues than not performing them. Before a new project deliverable can begin, the current deliverable must first be approved by a group of individuals who inspect the software for defects that can negatively impact the software qualities. Additionally, this group can also indicate areas in the software where the specified expectations and standards are being ignored where the developers have to make changes to the software match the missing standards and expectations. Therefore, reviews and inspections ensure that expectations continue to be met by removing software defects before it begins to accumulate and that the developers do not accidentally ignore these requirements.

The quality management process can be improved to address the issues identified during testing by adding additional conditions to the review and inspection aspect of the quality management process. These additional conditions should include identifying areas in the software where performance drops significantly, determine that potential errors are handled effectively and explained properly as well as analysing the security of each component of the software. Adding these conditions ensures that the issues identified during beta testing are already found and solved before the software is sent for testing, enabling for other hidden defects that were missed to be found.

3.2) The ISO 9001 standard is a framework that defines software standards by defining the quality principles and processes as well as the organisation's standards and procedures. The ISO/IEC 25010 standard is a flexible framework for quality assessment that identifies key characteristics of software and enables for consistent evaluations (Şahin, Tarhan, 2025).

The ISO 9001 standard can assist in quality management by determining how the organisation defines quality in software as well as the general quality processes and procedures utilised by the organisation. Creating consistent view on how quality is determined that should be followed by all personal within the organisation, improves quality management by reducing the potential confusion from developers on the desired qualities of the software where they can then effectively implement features that supports these software qualities. Furthermore, the ISO 9001 standard can be reused for multiple software projects to ensure consistency in aspects such as complying

with regulatory requirements, securing the software with encryption and authentication as well as ensuring reliability through fault tolerance mechanisms.

The ISO/IEC 25010 standard can improve quality management by identifying issues related to the software's reliability, security and compliance by performing consistent evaluations and quality assessments. Performing evaluations and quality assessments based on the key characteristics of the software increases the likelihood that any problems such as security vulnerabilities, performance drops and regulation violations are identified and solved during software development than not performing these evaluations. In addition, the consistent nature of these evaluations and quality assessments can prevent developers from accidently implementing features that do not meet the desired software qualities, which can improve the reliability, security and compliance qualities of the software.

Bibliography

Anonymous (2009) *5: Test analysis and design*. London: BCS Learning & Development Limited. [Online] Available at ProQuest

Bindu Bhargavi, S.M. and Suma, V. (2022) ‘A Survey of the Software Test Methods and Identification of Critical Success Factors for Automation’, *SN Computer Science*, 3(6), p. 449. [Online] Available at ProQuest

Csuhaj-Varjú, E. and Vaszil, G. (2024) ‘Variants of distributed reaction systems’, *Natural Computing*, 23(2), pp. 269–284. [Online] Available at ProQuest

Fatima, Z.T., Khtira, A. & Asri, B.E. (2024), "Algorithmic Business Process Optimization: Empowering Operational Excellence with Service-Oriented Architecture (SOA) and Microservices", *Ingenierie des Systemes d'Information,* vol. 29, no. 6, pp. 2399-2413. [Online] Available at ProQuest

Ian Sommerville, (2016), Software Engineering, Tenth Edition, 236

Leung, H.K.N. and Wong, P.W.L. (1997) ‘A study of user acceptance tests’, *Software Quality Journal*, 6(2), pp. 137–149. [Online] Available at ProQuest

Mafi, Z. and Mirian-Hosseinabadi, S. (2024) ‘Regression test selection in test-driven development’, *Automated Software Engineering*, 31(1), p. 9. [Online] Available at ProQuest

Malik, S. *et al.* (2019) ‘An Adaptive Emergency First Intelligent Scheduling Algorithm for Efficient Task Management and Scheduling in Hybrid of Hard Real-Time and Soft Real-Time Embedded IoT Systems’, *Sustainability*, 11(8), p. 2192. [Online] Available at ProQuest

Hassan, M.U., Mubashir, M., Shabir, M.A. & Ullah, M.M. (2018), "SOFTWARE QUALITY ASSURANCE TECHNIQUES: A REVIEW", *International Journal of Information, Business and Management,* vol. 10, no. 4, pp. 214-221. [Online] Available at ProQuest

Pandey, D. (2013) ‘The Research Roadmap of Requirement Validation’, *International Journal of Advanced Research in Computer Science*, 4(4). [Online] Available at ProQuest

Şahin, M.C. & Tarhan, A.K. (2025), "Evaluation and Selection of Hardware and AI Models for Edge Applications: A Method and A Case Study on UAVs", *Applied Sciences,* vol. 15, no. 3, pp. 1026. [Online] Available at ProQuest

Schoch, D.J. and Laplante, P.A. (1995) ‘A real-time systems context for the framework for informatio’, *IBM Systems Journal*, 34(1), p. 20. [Online] Available at ProQuest

Stejskal, R. (2014) *Test-Driven Learning in High School Computer Science*, *ProQuest Dissertations and Theses*. [Online] Available at ProQuest